

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Mikroelektroniki i Optoelektroniki

Praca dyplomowa inżynierska

na kierunku Elektronika
w specjalności Elektronika i Fotonika

Dedykowana aplikacja do analizy analogowych układów elektronicznych
w języku programowania Swift

Oleksandr Bolbat

Numer albumu 329915

promotor
mgr inż. Bartosz Dec

WARSZAWA 2026

1. Streszczenie Pracy

Praca inżynierska skupia się na napisaniu dedykowanej aplikacji do analizy analogowych układów elektronicznych w języku programowania Swift. Stworzona aplikacja służy do pokazania nadmiernego ignorowania komputerów z systemem operacyjnym macOS w dziedzinie projektowania oraz symulowania układów elektronicznych. Prace rozpoczęto od zebrania informacji dotyczących tworzenia aplikacji służących do analizy układów elektronicznych, sposobach implementacji metod numerycznych pozwalających na ich analizę oraz pogłębienia znajomości języka programowania Swift. Zakres pracy obejmował napisanie poszczególnych bloków aplikacji umożliwiających parsowania opisu układu ze zwykłego pliku tekstowego oraz tworzenie elementów na podstawie ich szablonów, obliczanie punktu pracy i analizę czasową układu przy użyciu metod numerycznych. W procesie zostały napisane zabezpieczenia chroniące od analizowania błędnych układów oraz testy sprawdzające poszczególne bloki aplikacji. Na końcu napisane rozwiązania zostały porównane z wcześniej istniejącymi względem szybkości i poprawności działania.

Abstract

The engineering thesis focuses on developing a dedicated application for analyzing analog electronic circuits using the Swift programming language. The application was created to demonstrate the excessive neglect of computers running on the macOS operating system in the field of electronic circuit design and simulation. The work began with gathering information on the development of applications for circuit analysis, methods of implementing numerical techniques enabling such analysis, and deepening knowledge of the Swift programming language. The scope of the project included implementing individual application modules enabling the parsing of a circuit description from a plain text file and creating components based on their templates, calculating the operating point, and performing time-domain analysis using numerical methods. During the process, safeguards were implemented to prevent the analysis of incorrect circuits, along with tests verifying individual modules of the application. Finally, the developed solutions were compared with previously existing ones in terms of speed and correctness.

Spis treści

1.	Streszczenie pracy.....	2
	Abstract.....	3
2.	Wstęp.....	6
3.	Cel i zakres pracy.....	7
4.	Podstawy teoretyczne analizy obwodów.....	8
4.1.	Równania obwodów.....	8
4.2.	Metody macierzowe w analizie obwodów.....	9
4.3.	Klasyczna metoda potencjałów węzłowych.....	9
4.4.	Zmodyfikowana metoda potencjałów węzłowych.....	10
4.5.	Punkt pracy DC.....	11
4.6.	Analiza czasowa.....	11
5.	Założenia i architektura aplikacji.....	12
6.	Implementacja macierzy.....	14
6.1.	Struktura i implementacja klasy Matrix().....	14
6.2.	Zabezpieczenia i obsługa błędów.....	14
6.3.	Testowanie.....	15
6.4.	Napotkane problemy implementacyjne.....	16
6.5.	Porównanie szybkości działania.....	16
7.	Parser netlisty.....	17
8.	Zmodyfikowana metoda potencjałów węzłowych.....	17
8.1.	Szablony elementów.....	18
8.2.	Wypełnianie macierzy globalnej.....	19
8.3.	Walidacja parametrów i zabezpieczenia w szablonach.....	21
8.4.	Testowanie.....	21
9.	Obliczenie punktu pracy.....	21
9.1.	Założenia i budowa układu równań.....	21
9.2.	Walidacja i obsługa błędów.....	22
9.3.	Testowanie.....	22
10.	Analiza czasowa.....	22

10.1.	Założenia.....	22
10.2.	Dyskretyzacja elementów dynamicznych.....	23
10.3.	Konstrukcja układu równań.....	24
10.4.	Pętla symulacji czasowej.....	24
10.5.	Stabilność i zabezpieczenia.....	24
10.6.	Prezentacja wyników i dane wyjściowe.....	25
10.7.	Testowanie.....	25
11.	Podsumowanie.....	28
	Bibliografia.....	29
	Wykaz symboli i skrótów.....	30
	Spis rysunków.....	31

2. Wstęp

Początkowo, przez długi czas, układy elektroniczne rysowane były odręcznie i wszelkie obliczenia związane z ich działaniem wykonywane bezpośrednio przez projektantów za pomocą prostych kalkulatorów. Proces ten był czasochłonny, niewygodny oraz niedokładny, co wprowadzało wiele możliwości występowania błędów. Z tych powodów szukano rozwiązań optymalizujących obliczenia.

Pierwsze oprogramowanie do analizy obwodów powstały już w latach 60. i 70. XX wieku. Jeden z nich to ECAP¹, opracowany w 1965 roku program użytkowy do analizy układów elektronicznych. Innym przykładem jest SPICE² z 1973 roku, powstały na Uniwersytecie Kalifornijskim w Berkeley, uznawany za fundament współczesnych symulatorów obwodów analogowych.

Od tamtego czasu programy symulacyjne stawały się coraz powszechniej użytkowane co prowadziło do ich rozwoju, powstawania wielu różnych rozwiązań oraz ulepszeń. Przez długi okres jedynym systemem operacyjnym stosowanym do takich celów był Windows, jednak wraz ze wzrostem popularności macOS pojawił się niedobór na rynku. Większość dotychczasowych programów oficjalnie nie uwzględnia w pełni urządzeń firmy Apple. Użytkownicy zmuszeni są do stosowania z wirtualizacji, zdalnych środowisk czy alternatywnych, mniej dopracowanych zamienników. Nie pozwala to na wygodną pracę i stwarza problem zmiany urządzeń lub korzystania z różnych wątpliwej jakości rozwiązań. Zdarza się, że producenci wspominają w dokumentacji o uruchamianiu ich programów na Macu, ale jak na przykładzie Altium Designer, tylko za pomocą konfiguracji pośrednich, czyli wirtualizacji³. W innych przypadkach występuje deklaracja dystrybucji tylko jako produkt wspierający system Windows np. Multisim.

Komputery Apple w ostatnich latach stały się komfortową platformą do zadań inżynierskich, szczególnie biorąc pod uwagę układy Apple Silicon. W zaletach ich wykorzystania można uwzględnić między innymi wysoką wydajność energetyczną czy znaczące możliwości obliczeń numerycznych. Potwierdzają to współczesne analizy wydajnościowe układów serii M w zadaniach obliczeniowych (m.in. w kontekście relacji osiągnięć do poboru mocy⁴). Dodatkowo wyróżnić można spójny ekosystem programistyczny macOS oraz języka Swift, który umożliwia tworzenie natywnych aplikacji o wysokiej wydajności i dobrej integracji z systemem.

Innym napotkanym problemem może być ograniczony dostęp do aktualnych zasobów. Profesjonalne narzędzia do symulacji i projektowania typowo wykorzystywane na rynku są przeważająco oprogramowaniem zamkniętym. Ogranicza to możliwość wglądu w implementację metod obliczeniowych, ich modyfikowania oraz dopasowywania do nietypowych zastosowań. Dodatkowo może to powodować wykluczenie większości indywidualnych użytkowników pracujących nad indywidualnymi projektami ze względu na konieczność najlepszych rozwiązań, które dostosowane są do dużych firm i często w postaci subskrypcji. W związku z tym zmuszeni do korzystania z aplikacji darmowych powoduje poleganie na programach często starych⁵ i nieoptymalnych, czyli niedostosowanych do bardziej skomplikowanych i dokładnych symulacji.

Rozważając powyższe argumenty, można zauważyć potrzebę utworzenia możliwości przystępnego, aktualnego i otwartego oprogramowania do przeprowadzania symulacji układów elektronicznych skierowanego dla użytkowników urządzeń Apple. Z tego powodu w niniejszej pracy wykonana zostanie aplikacja do analizy analogowych obwodów elektronicznych, opartej na ZMPW oraz

¹ [1] An Annotated Bibliography of Computer-Aided Circuit Analysis and Design

² [2] Simulation Program with Integrated Circuit Emphasis (SPICE)

³ [3] KB: Run Altium on non-Windows platform

⁴ [4] Apple vs Oranges: Evaluation Apple Silicon M-Series SoCs for HPC performance and Efficiency

⁵ [5] Wikipedia: Ngspice

koncepcji szablonów elementów. Skupi się ona na pokazaniu możliwości rozwiązania problemów z brakiem dostępności podobnych programów na macOS.

3. Cel i zakres pracy

Cel pracy polegał na napisaniu dedykowanej aplikacji do symulacji analogowych układów elektronicznych w języku programowania Swift przeznaczonej dla komputerów z systemem operacyjnym macOS. Napisana aplikacja jest dowodem możliwości i wygody tworzenia narzędzi służących do symulowania obwodów elektronicznych na platformę Apple z punktu widzenia programisty. Głównym założeniem jest wykorzystanie spójnej integracji języka programowania Swift z ekosystemem Apple oraz potencjału obliczeniowego współczesnych komputerów Apple, co jest krytyczne dla symulacji obwodów. Dodatkowo poszczególne elementy aplikacji miały być realizowane w przejrzysty i łatwy do skalowania sposób.

Zakres pracy obejmował zaprojektowanie oraz implementację kluczowych elementów aplikacji, zaczynając od wczytania opisu układu z postaci pliku tekstowego do wykonania obliczeń i uzyskania wyników analizy. W ramach pracy zostały zrealizowane poszczególne elementy:

- Parser netlisty - moduł służy do wczytania układu z postaci zbliżonej do netlist SPICE, sprawdzenie poprawności opisanego układu oraz przekształcenie go do postaci wewnętrznej.
- Implementacja macierzy - opracowanie klasy służącej do przechowywania i obsługi macierzy.
- Szablony elementów - stworzenie zestawu szablonów podstawowych elementów analogowych w sposób łatwy do skalowania oraz użycia podczas wypełniania macierzy na podstawie ZMPW. Stworzone szablony zachowują informację potrzebną do poprawnego działania parsera netlisty i zabezpieczenia od tworzenie niepoprawnych elementów.
- Blok wypełniający macierze - implementacja mechanizmu, który umożliwi łatwe wypełnienie macierzy na podstawie listy elementów.
- Obliczanie punktu pracy - realizacja obliczeń stanu ustalonego, służących do weryfikacji poprawności zbudowanego układu oraz będącego punktem początkowym dla dalszych analiz.
- Analiza czasowa - implementacja analizy czasowej, umożliwiającej wyznaczenie odpowiedzi układu w funkcji czasu.
- Zabezpieczenia i obsługa błędów - dodanie mechanizmów zabezpieczających część obliczeniową od błędnych danych wejściowych i niepoprawnego działania aplikacji. Służy do opisu miejsca oraz kontekstu powstałego błędu.

Głównym celem pracy było zaprezentowanie kompletnej ścieżki działania aplikacji oraz opisanie rozwiązań i napotkanych problemów podczas projektowania i implementacji elementów, umożliwiających zbudowanie narzędzia do symulacji na komputerach z systemem operacyjnym macOS.

4. Podstawy teoretyczne analizy obwodów

4. 1. Równania obwodów

W ramach symulacji układu powstaje potrzeba jego opisu, co umożliwi jego dalsze przetwarzanie. Podstawą takiego opisu są prawa Kirchhoffa⁶:

- Pierwsze prawo Kirchhoffa - suma natężeń prądów wpływających jest równa sumie natężeń prądów wypływających (1)

$$\sum_k i_k = 0 \quad (1)$$

Dla węzła o potencjale v , każdy dołączony do węzła element dodaje do równania zależność prądu od napięcia węzłowych.

- Drugie prawo Kirchhoffa - skierowana suma różnic potencjałów w dowolnej zamkniętej pętli wynosi zero (2)

$$\sum_k u_k = 0 \quad (2)$$

KVL jest szczególnie naturalne w metodach oczkowych, natomiast w analizie węzłowej jest ono ukryte w konstrukcji równań poprzez zależności elementów i dobór niewiadomych.

W obwodach liniowych i liniowo aproksymowanych fragmentach obwodów nieliniowych są wykorzystywane charakterystyki elementów w postaci równań liniowych, jak na przykład⁷:

- Rezystor między węzłami a i b o rezystancji R (3)

$$i_{ab} = \frac{u_a - u_b}{R} = G(u_a - u_b), \quad G = \frac{1}{R} \quad (3)$$

- Niezależne źródło prądowe wymusza zadany prąd I . W zapisie KCL zapisuje się w postaci składnika stałego w wektorze wymuszeń.
- Niezależne źródło napięciowe dodaje różnice potencjałów (4)

$$u_a - u_b = V \quad (4)$$

Taki element jest problematyczny w klasycznej MPW, ponieważ nie jest możliwy bezpośredni opis jako prąd zależny od napięć węzłowych.

W obwodach analogowych często występują także elementy sterowane, jak na przykład źródła sterowane⁸:

- Źródło prądowe sterowane napięciem (5)

$$i = g(v_c - v_d) \quad (5)$$

- Źródło napięciowe sterowane napięciem (6)

$$v_{ab} = \mu(v_c - v_d) \quad (6)$$

⁶ [6] Wikipedia: Kirchhoff's Circuit Laws

⁷ [7] A system approach to circuits, measurements and control, Rozdział 3.3.1

⁸ [7] A system approach to circuits, measurements and control, Rozdział 3.3.2

- Źródło prądowe sterowane prądem (7)

$$i = \beta i_x \quad (7)$$

- Źródło napięciowe sterowane prądem (8)

$$v_{ab} = r i_x \quad (8)$$

Obsługa źródeł napięciowych, z powodów opisanych w rozdziale 4. 4., powodują konieczność użycia ZMPW.

4. 2. Metody macierzowe w analizie obwodów

Symulacje komputerowe wymagają doprowadzenia układu do postaci macierzowej. Niezależnie od wybranej metody, wynikiem takich doprowadzeń, będzie układ równań liniowych (9)

$$Ax = z \quad (9)$$

gdzie:

- A – macierz współczynników. Wynik połączeń i parametrów elementów,
- x – wektor niewiadomych. Napięcia węzłowe, prądy przez źródła,
- z – wektor wymuszeń. Źródła niezależne i/lub składniki zależne od warunków początkowych.

W obwodach liniowych taki układ równań liniowych rozwiązywany jest standardowymi metodami algebry liniowej, jak na przykład eliminacja Gaussa czy rozkład LU. W przypadku analizy czasowej lub przy elementach dynamicznych, jak na przykład kondensator i cewka, pojawia się zależność od czasu, co prowadzi do równań różniczkowych, które po dyskretyzacji również przybierają postać układów liniowych.

4. 3. Klasyczna metoda potencjałów węzłowych

Klasyczna metoda potencjałów węzłowych⁹ polega na przyjęciu węzłów względem węzła odniesienia, czyli masy, jako niewiadomych. Następnie dla każdego węzła, pomijając masę, zapisuje się KCL wyrażając prądy przez elementy w funkcji różnic potencjałów.

W przypadku, gdy obwód składa się z elementów admitancyjnych, jak na przykład rezystor czy kondensator po dyskretyzacji, oraz źródeł prądowych, możliwe jest zbudowanie macierzy konduktancji G spełniającej równanie (10):

$$Gv = i \quad (10)$$

gdzie:

- v – wektor napięć węzłowych,
- i – wektor prądów wstrzykiwanych do węzłów przez źródła prądowe.

Macierz G ma charakterystyczną strukturę:

- elementy diagonalne G_{kk} są sumą konduktancji dołączonych do węzła k ,

⁹ [8] Wikipedia: Nodal Analysis

- elementy pozadiagonalne G_{km} są ujemną sumą konduktancji łączących węzły k i m .

Ograniczenie klasycznej MPW ujawnia się przy elementach, które nie są opisane admitancją, jak na przykład:

- Idealne źródło napięciowe
- Elementy narzucające zależności napięciowe

W takich przypadkach układ równań nie jest możliwy do zapisania w postaci (10) bez wprowadzenia sztucznych rezystancji, superwęzłów lub dodatkowych zmiennych. To powoduje przejście do ZMPW.

4. 4. Zmodyfikowana metoda potencjałów węzłowych

Zmodyfikowana metoda potencjałów węzłowych¹⁰ rozszerza klasyczną MPW o dodatkowe niewiadome, jak na przykład prądy wymuszane przez idealne źródła napięciowe czy zależne źródła napięciowe. To powoduje:

- Brak konieczności eliminowania źródeł napięciowych,
- Łatwiejszą obsługę elementów sterowanych,
- Zachowanie struktury umożliwiającej dodawanie wkładów elementów do macierzy.

Przykładowy wektor niewiadomych ZMPW ma postać (11)

$$x = \begin{bmatrix} v \\ i_s \end{bmatrix} \quad (11)$$

gdzie:

- v – napięcia węzłowe. Masa jest pomijana,
- i_s – prądy przez wybrane elementy pomijane klasyczną metodą

Ogólny układ równań ZMPW można zapisać blokowo (12)

$$\begin{bmatrix} G & B \\ C & D \end{bmatrix} \begin{bmatrix} v \\ i_s \end{bmatrix} = \begin{bmatrix} I \\ E \end{bmatrix} \quad (12)$$

Bloki układu równań ZMPW:

- G – macierz konduktancji wynikająca z elementów powodujących zależność prąd-napięcie, jak na przykład rezystor,
- B – macierz wiązań pomiędzy węzłami a dodatkowymi prądami, jak na przykład podłączenia źródeł napięciowych do węzłów,
- C – często $C = B^T$ dla źródeł niezależnych. Dla elementów sterowanych mogą pojawić się inne zależności,
- D – blok „dodatkowych” równań, zerowy dla idealnych źródeł napięciowych,
- I – wektor wymuszeń prądowych, spowodowanych źródłami prądowymi,

¹⁰ [9] Wikipedia: Modified nodal analysis

- E – wektor wymuszeń napięciowych. Wartości źródeł napięciowych lub równania narzucone przez elementy sterowane.

Przykładowo idealne źródło napięciowe między węzłami a i b o wartości V w ZMPW dodaje niewiadomą i_v - prąd przez źródło oraz równanie (13)

$$v_a - v_b = V \quad (13)$$

W macierzy reprezentowano to jako dodanie wartości w wierszach odpowiadających v_a , v_b oraz nowej zmiennej i_v do wektora niewiadomych. Taki szablon jest bardzo łatwy do implementacji i dalszego użycia.

Element sterowane:

- Źródło prądowe sterowane napięciem można wprowadzić bez dodatkowych niewiadomych wnosząc odpowiednie współczynniki do równań KCL w węzłach.
- Źródło napięciowe sterowane napięciem wymaga dodatkowej niewiadomej prądowej analogicznie do źródła napięciowego niezależnego, ale równanie narzucone ma postać (14)

$$v_a - v_b = \mu(v_c - v_d) \quad (14)$$

co generuje dodatkowy wiersz i dodaje niewiadomą do wektor niewiadomych

To prowadzi do zalety ZMPW - ujednolicenie postaci matematycznej dla szerokiej klasy elementów przy zachowaniu prostego procesu budowania macierzy przy użyciu szablonów elementów.

4. 5. Punkt pracy DC

W symulatorze obwodów takie macierzy i wektory są wykorzystywane w różnych trybach analizy. Najczęściej spotykane są: obliczanie punktu pracy oraz analiza czasowa.

Obliczanie punktu pracy¹¹ polega na znalezieniu stanu ustalonego obwodu dla stałych wymuszeń. Dla układów liniowych sprowadza się to do rozwiązania jednego układu (9)

W przypadku obwodów zawierających elementy dynamiczne:

- Kondensator w stanie ustalonym zachowuje się jak przewa, ponieważ prąd stały płynący przez idealny kondensator wynosi zero,
- Cewka w stanie ustalonym zachowuje się jak zwarcie, ponieważ spadek napięcia na idealnej cewce wynosi zero.

W praktyce symulator może:

- Pomijać wkłady dynamiczne podczas obliczania punktu pracy,
- Stosować uogólniony model, jeśli w dalszych obliczeniach przewidywana jest obsługa elementów nieidealnych.

Dla obwodów nieliniowych, jak na przykład dioda czy tranzystor, punkt pracy wymaga iteracji, jednak w zakresie pracy można ograniczyć się do przypadków liniowych lub do uproszczonych modeli.

¹¹ [10] Zaawansowane metody symulacji układów elektronicznych, Rozdział 5.2

4. 6. Analiza czasowa

Analiza czasowa¹² opisuje odpowiedź układu w funkcji czasu. Takie elementy jak kondensator czy cewka prowadzą do równań różniczkowych. Postać równań w ZMPW dla układów liniowych dynamicznych jest zapisana w postaci (15)

$$C \frac{dx}{dt} + Gx = b(t) \quad (15)$$

gdzie:

- x zawiera napięcia węzłowe i ewentualne prądy dodatkowe,
- C to macierz „pojemnościowo-indukcyjna”,
- G odpowiada części rezystancyjnej,
- $b(t)$ to wymuszenia zależne od czasu.

W ramach rozwiązania problem numerycznych, stosowana jest dyskretyzacja czasu - wybiera się krok h i wyznacza się przybliżenie pochodnej. Dwie klasyczne metody całkowania używane w symulacji obwodów to:

- Interpolacyjna metoda Eulera (16)

$$\left. \frac{dx}{dt} \right|_{t_k} \approx \frac{x_k - x_{k-1}}{h} \quad (16)$$

Po podstawieniu do równania:

$$C \frac{x_k - x_{k-1}}{h} + Gx_k = b_k \quad (17)$$

co można przekształcić do postaci układu liniowego:

$$(G + \frac{C}{h})x_k = b_k + \frac{C}{h}x_{k-1} \quad (18)$$

Zaletą jest duża stabilność numeryczna.

- Metoda trapezów (19)

$$\left. \frac{dx}{dt} \right| \approx \frac{x_k - x_{k-1}}{h}, \quad Gx \text{ i } b(t) \text{ uśredniane} \quad (19)$$

W efekcie otrzymuje się układ:

$$(G + \frac{2C}{h})x_k = b_k + b_{k-1} + (\frac{2C}{h} - G)x_{k-1} \quad (20)$$

Metoda trapezów jest dokładniejsza, ale również może generować oscylacje numeryczne w pewnych warunkach.

W obu przypadkach analiza czasowa sprowadza się do wielokrotnego rozwiązywania układu równań liniowych dla kolejnych kroków czasowych t_k co powoduje:

- Potrzebę wydajnej implementacji macierzy,

¹² [10] Zaawansowane metody symulacji układów elektronicznych, Rozdział 7.1

- Stabilnego modułu do rozwiązywania równań liniowych,
- Potrzebę dodania mechanizmów kontroli błędów i zabezpieczeń.

5. Założenia i architektura aplikacji

Projektowana aplikacja została podzielona na niezależne bloki funkcjonalne, które tworzą spójny łańcuch przetwarzania - od wczytania opisu układu do uzyskania wyników symulacji. Taki podział jest celowy i niezbędny - pozwala utrzymać przejrzystość kodu, ogranicza zależności pomiędzy częściami systemu oraz ułatwia zarówno dalszą rozbudowę programu, naprawę błędów i testowanie poszczególnych modułów niezależnie. W kolejnych rozdziałach każdy z opisanych bloków zostanie omówiony szczegółowo.

Parser netlisty

Pierwszym modułem w aplikacji występuje parser netlisty, element odpowiedzialny za wczytanie pliku tekstowego z opisem obwodu w formacie zbliżonym do SPICE¹³ oraz przekształcenie go do postaci wewnętrznej. Wynikiem pracy parsera jest lista elementów wraz z ich parametrami, informacją o połączeniach. Wydzielenie parsera jako osobnego bloku aplikacji pozwala na niezależne rozwijanie składni wejściowej, dodawanie nowych dyrektyw lub formatów bez zmian części obliczeniowej.

Blok ZMPW i wypełnianie macierzy szablonami elementów

Kolejny modułem występuje część odpowiedzialna za zbudowanie układu równań metodą ZMPW. W tym projekcie krytyczne jest wypełnianie macierzy oraz wektorów poprzez sumowanie lokalnych wkładów szablonów elementów. Każdy element obwodu dostarcza własny szablon opisujący, jakie współczynniki powinny zostać dodane do macierzy i wektora wymuszeń. To podejście umożliwia proste skalowanie - dodanie nowego elementu sprowadza się do implementacji jego szablonu bez przebudowy całego mechanizmu tworzenia równań.

Obliczanie punktu pracy

Po zbudowaniu równań aplikacja wykonuje obliczenie punktu pracy, czyli stanu ustalonego układu dla wymuszeń stałych. Ten blok odpowiada za przygotowanie odpowiednich macierzy dla przypadku DC oraz rozwiązanie układu równań w celu wyznaczenia napięć węzłowych oraz dodatkowych niewiadomych wynikających z ZMPW. Wynik punktu pracy stanowi podstawową weryfikację poprawności modelu oraz punkt startowy dla kolejnych analiz.

Analiza czasowa

Ostatnim z głównych bloków jest analiza czasowa, która wyznacza odpowiedź układu w kolejnych chwilach czasu. Moduł ten realizuje iteracyjne obliczenia w krokach czasowych, wykorzystując mechanizmy dyskretyzacji elementów dynamicznych oraz wielokrotne rozwiązywanie kolejnych układów równań. Wydzielenie analizy czasowej jako osobnej części pozwala niezależnie rozwijać metody całkowania, kontrolę kroku czasowego oraz sposób prezentacji wyników, bez wpływu na inne bloki aplikacji.

¹³ [2] Simulation Program with Integrated Circuit Emphasis(SPICE)

6. Implementacja macierzy

W analizie obwodów metodą ZMPW centralnym elementem obliczeń jest rozwiązywanie układów równań liniowych oraz wielokrotne modyfikowanie macierzy współczynników i wektorów wymuszeń. Z tego względu przed implementacją parsera netlisty oraz bloków analizy zdecydowano się przygotować niezależny moduł macierzowy, który zapewnia spójny interfejs do tworzenia, wypełniania i rozwiązywania macierzy wykorzystywanych w dalszych etapach aplikacji.

6. 1. Struktura i implementacja klasy Matrix()

Macierz została zaimplementowana jako klasa Matrix. Wybór klasy (a nie struktury) wynikał przede wszystkim z wygody w pracy z referencjami i możliwości transformacji danych podczas budowania globalnych układów równań. W praktyce pozwala to łatwiej przekazywać macierz pomiędzy blokami oraz modyfikować ją bez konieczności częstego kopiowania.

Obiekt klasy Matrix przechowuje:

- Wymiary macierzy - ilość wierszy oraz kolumn,
- Wartości elementów w postaci jednowymiarowej tablicy [Double],
- Metody pomocnicze umożliwiające operacje wymagane przez ZMPW.

Realizowane podstawowe funkcje:

- show() - metoda pozwala wyświetlić macierz w czytelnej formie. Metoda używana podczas testowania oraz debugowania,
- add() - metoda pozwalająca dodawać wartości do wybranych komórek. Metoda została realizowana w trzech wersjach: dodanie pojedynczej wartości do macierzy, dodanie pojedynczej wartości do wektora oraz dodawanie całej macierzy,
- divide() - metoda rozwiązująca układ równań. Wykorzystywana jest do obliczeń równań postaci (9).

Implementacja macierzy jako osobnego modułu pozwala na jej testowanie, rozwinięcie bez wpływu na inne moduły.

6. 2. Zabezpieczenia i obsługa błędów

Macierze w aplikacji są wypełniane automatycznie na podstawie szablonów elementów, co powoduje pewne ryzyko: do macierzy mogą trafić dane niespójne lub wskazujące na błąd w konstrukcji równań. Kluczowe jest wykrycie takich sytuacji. W tym celu został dodany zestaw zabezpieczeń, które chronią przed pomyłkami i ułatwiają diagnozowanie problemów.

Zastosowane mechanizmy:

- Dodawanie do macierzy wartości nil, czyli braku danych, jest ignorowane. Umożliwia to implementację iteracji szablonów w łatwiejszy sposób - każdy element próbuje dodać do macierzy konduktancji oraz wektora niewiadomych dane. W przypadku gdy element nie musi mieć wpływu na macierz czy wektor, wysyłane dane są puste, w wyniku są pomijane,
- Dodawanie wartości równej zero jest ignorowane. Ponieważ dodanie zera nie ma wpływu, ten proces jest pomijany,
- Próba dodania wartości poza zakresem wymiarów macierzy powoduje błąd. Ponieważ dodanie wartości poza zakresem może być spowodowane wyłącznie błędem w implementacji

szablonu elementu czy procesie tworzenia macierzy konduktancji i/lub wektora niewiadomych, konieczne jest zgłoszenie błędu wraz z opisem niepoprawności operacji.

- Dodawanie macierzy o niezgodnych rozmiarach powoduje błąd. Powodem takiej operacji może być wyłącznie błąd implementacji szablonu czy tworzenia macierzy konduktancji i/lub wektora niewiadomych. Z tego powodu taka operacja musi być zgłoszona jako błąd wraz z jej opisem,
- `divide()` na pustą macierz powoduje błąd. Rozwiązywanie układu równań bez poprawnie przygotowanej macierzy jest sytuacją krytyczną, co powoduje konieczność zgłoszenia błędu,
- `divide()` przy niepoprawnych wymiarach. Metoda `divide()` wymaga kwadratowej macierzy o rozmiarach $N \times N$ oraz wektora o rozmiarze N . W przypadku gdy dowolny z tych warunków nie jest spełniony, konieczne jest zgłoszenie błędu,
- Brak możliwości rozwiązania układu równań powoduje błąd. Brak możliwości rozwiązania układu jest interpretowany jako sygnał, że wartości z szablonów elementów zostały zebrane w niepoprawny sposób. Z tego powodu sytuacja ta nie jest traktowana jako poprawna ścieżka działania, tylko jako błąd, który powinien zostać wykryty i zgłoszony.

Dodanie takich zabezpieczeń umożliwia wykrycie problemów w logice i budowy równań.

6. 3. Testowanie

Moduł macierzowy jest najbardziej krytycznym elementem aplikacji, ponieważ błędy w operacjach na macierzach mogą prowadzić do niepoprawnych wyników symulacji lub trudnych do zdiagnozowania problemów numerycznych. Z tego powodu został przygotowany zestaw testów jednostkowych, których celem jest weryfikacja poprawności działania podstawowych operacji oraz mechanizmów zabezpieczeń.

Zakres testów obejmuje:

- Tworzenie obiektów macierzy - sprawdzane jest, czy obiekt klasy `Matrix` przechowuje poprawne wartości liczby wierszy i kolumn oraz czy wewnętrzna tablica danych ma zgodny rozmiar,
- Testowanie funkcji `add()` - sprawdzany jest proces dodawanie poszczególnych wartości oraz macierzy. Test potwierdza ignorowanie dodawania pustych danych,
- Testy zabezpieczeń indeksów i rozmiarów - Sprawdzane jest zgłaszanie błędów w sytuacjach takich jak:
 - Próba dodania wartości poza zakresem macierzy,
 - Próba wykonania operacji na macierzach o niezgodnych rozmiarach.
- Testy funkcji `divide()` - weryfikowana jest obsługa błędów oraz warunków poprawności:
 - Zgłoszenie błędu przy próbie rozwiązania układu z macierzą nil,
 - Zgłoszenie błędu dla niepoprawnych kształtów danych,
 - Poprawność wyniku dla prostych, znanych układów równań, gdzie rozwiązanie można łatwo zweryfikować analitycznie,
 - Wykrywanie przypadków, w których implementacja nie jest w stanie rozwiązać układu i traktowanie tego jako błąd logiczny w kontekście symulacji.

Testy umożliwiają dalszy modyfikację projektu, jak na przykład wprowadzenie optymalizacji czy zmiana sposobu przechowywania danych, ponieważ wprowadzone zmiany mogą być szybko zweryfikowane poprzez uruchomienie zestawu testów.

6. 4. Napotkane problemy implementacyjne:

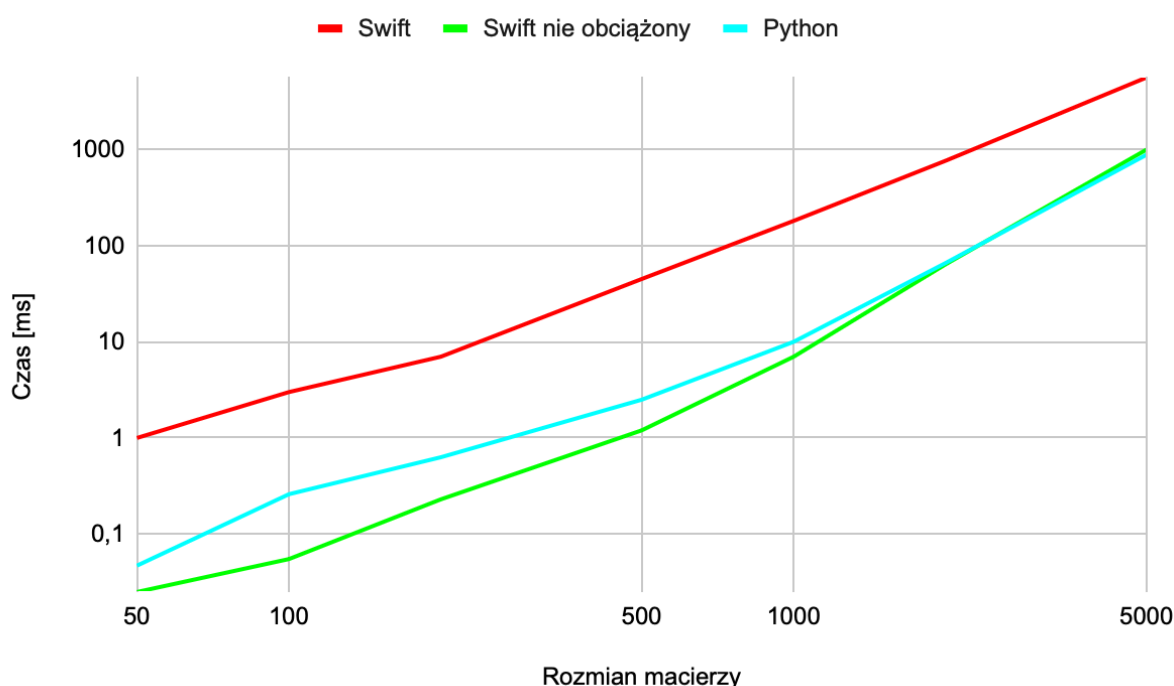
W ramach implementacji rozwiązywania układów równań została wykorzystana funkcja `dgesv_(LAPACK)`¹⁴, która realizuje rozwiązywanie układ równań postaci (9) metodą rozkładu LU z wyborem elementu głównego. W trakcie integracji ujawniono krytyczny problem związany z formatem przechowywanych danych:

Podczas implementacji rozwiązywania układów równań wykorzystano funkcję `dgesv_ (LAPACK)`, która realizuje rozwiązanie $Ax=b$ metodą rozkładu LU z wyborem elementu głównego. W trakcie integracji ujawniono istotny problem związany z formatem przechowywania danych:

- `dgesv_()` oczekuje macierzy zapisanej w porządku column-major (kolumnami),
- natomiast naturalna implementacja w projekcie i sposób indeksowania w wielu fragmentach kodu była oparta o row-major (wierszami).

Zamiast przepisywać całą klasę `Matrix` oraz wszystkie miejsca jej zużycia na przechowywanie column-major (co wymagałoby zmian w indeksowaniu, testach i szablonach elementów), zdecydowano się na rozwiązanie kompromisowe: przed wywołaniem `dgesv_()` wykonywana jest odpowiednia transformacja danych do formatu oczekiwanego przez LAPACK, a po obliczeniach wynik jest mapowany z powrotem na format używany w aplikacji. Podejście to pozwoliło zachować spójność kodu w pozostałych blokach, kosztem kontrolowanego narzutu podczas samego rozwiązywania układu.

6. 5. Porównanie szybkości działania



Rys. 1 - Porównanie szybkości rozwiązywania układu równań.

¹⁴ [11] Solving systems of linear equations with LAPACK

Rysunek 1 pokazuje porównanie szybkości działań zaimplementowanych rozwiązań układu równań w trzech przypadkach:

- Swift - końcowa implementacja macierzy wraz z zabezpieczeniami i transformacją danych przed wywołaniem `dgesv_()`,
- Swift nie obciążony - implementacja bez zabezpieczeń i transformacji danych
- Python - implementacja w języku programowania Python

Wyniki pokazują potwierdzenie szybkiego działania zaimplementowanego rozwiązywania układu równań. Ponieważ programowanie na platformę Apple pozwala na użycie bardziej niskopoziomowych języków, jak na przykład C, do porównywania szybkości działania został wybrany Python, zamiast bardziej niskopoziomowego języka programowania.

7. Parser netlisty

Parser netlisty stanowi pierwszy etap działania aplikacji i odpowiada za przekształcenie wejściowego opisu układu w postaci pliku tekstowego na ustrukturyzowaną reprezentację wewnętrzną, wykorzystywaną w kolejnych blokach obliczeniowych.

Przyjęto założenie, że netlisty używane w aplikacji będą niewielkie, z tego powodu pliki są wczytywane w całości do pamięci, a następnie przetwarzane. Takie podejście upraszcza implementację, ułatwia wykonywanie wstępnej walidacji oraz pozwala na wygodne operacje na tekście, jak na przykład usuwanie komentarzy, podział na sekcje.

Po wczytaniu zawartości pliku tekst jest dzielony na linie, a następnie jest sprawdzane, któremu typowi elementu czy dyrektywie odpowiada opis. W przypadku opisu elementu, zostaje definiowany element oraz stworzenie jego kopii poprzez wywołanie konstruktora tego elementu z podaniem tej linii. Po sprawdzeniu poprawności opisu elementu, zwracana jest zbudowana kopia elementu i wstawiana do listy symulowanych elementów. W ten sposób parser kontroluje przepływ opisu elementów podczas ich tworzenia, natomiast składnie poszczególnych elementów przetwarzają te opisy, co ułatwia proces dodawania nowych szablonów.

Istotną częścią działania parsera jest walidacja netlisty i wykrywanie błędów w wejściowym opisie obwodu. Sprawdzane są powszechne błędy:

- Niezamknięta netlista. Sytuacja, w której opis układu jest niekompletny lub zawiera elementy bez pełnych danych,
- Brak wymaganej sekcji lub dyrektywy określającej rodzaj analizy,
- Brak znacznika końca opisu obwodu.

Wynikiem pracy parsera jest lista elementów gotowa do iterowania do wypełnienia macierzy konduktancji i wektorów. W przypadku wystąpienia błędu przetwarzania pliku wejściowego, parser zwraca odpowiedni błąd opisujący wykryty problem.

Zaprojektowany moduł pozwala na wygodne wczytanie danych wejściowych oraz filtrowanie błędów wejściowych przed rozpoczęciem kosztownych obliczeń.

8. Zmodyfikowana metoda potencjałów węzłowych

Po etapie parsowania danych wejściowych wszystkie elementy obwodu są zebrane do listy, gotowej do iterowania w ramach wypełnienia macierzy konduktancji i wektorów. Kolejny blok aplikacji odpowiada za zbudowanie układu równań przy użyciu ZMPW w postaci macierzowej. Cały proces realizowany na podstawie szablonów elementów elektrycznych opisujących wartości wpisywanych do

macierzy konduktancji i wektora wymuszeń. Tak zbudowany układ równań sprowadza się do iteracji po liście elementów i sumowania ich wkładów.

8. 1. Szablony elementów:

Szablonem elementu nazywamy klasę zachowującą połączenie, wartości parametrów oraz informacje o miejscu wpisywania dodatkowych wierszy, jak na przykład w przypadku źródła napięciowego.

Łatwe iterowanie po liście elementów jest możliwe, ponieważ każdy gotowy szablon dziedziczy po jednej głównej klasie zawierającej dwie funkcje: `getGMatrix()` oraz `getIMatrix()`. W ramach analizy czasowej dane funkcji można przyjmować jako parametr "krok czasowy".

Poniżej zaprezentowane realizacje szablonów rezystora i źródła napięciowego sterowanego prądem.

Rezystor pomiędzy węzłami a i b o rezystancji R

Dla rezystora obowiązuje (3)

W ZMPW rezystor wnosi wkłady wyłącznie do bloku G , czyli części napięciowo-węzłowej, macierzy konduktancji. Szablon ma postać jak zaznaczono na rysunku 2.

$$\begin{array}{l} \text{wiersz } a \rightarrow \\ \vdots \\ \text{wiersz } b \rightarrow \end{array} \begin{bmatrix} G & \cdots & -G \\ \vdots & \ddots & \vdots \\ -G & \cdots & G \end{bmatrix}$$

Rys. 2 - Szablon rezystora

W przypadku gdy dowolny węzeł jest dołączony do masy, odpowiednie wpisy związane z tym węzłem są pomijane.

Źródło napięciowe sterowane prądem pomiędzy węzłami w_{e+} , w_{e-} , w_{y+} i w_{y-} o transrezystancji r

Źródło napięciowe sterowane prądem opisuje zależność (8)

W implementacji ZMPW, aby obsłużyć takie źródło, wprowadza się:

- Dodatkowa niewiadoma prądowa i_{wy} , czyli prąd przez źródło,
- Równanie narzucające napięcie na zaciskach źródła zależne od i_{we}

Wtedy szablon źródła rozkłada się na dwa typy wpisów:

- Wpisy łączące i_{wy} z węzłami w_{y+} i w_{y-} ,
- Wpis zależny od i_{we} , który pojawia się w równaniu napięciowym źródła.

Na podstawie (12):

- Sprężenia w równaniach KCL w węzłach wy_+ i wy_- wynikające z prądu i_{wy} wpisywane do bloku B,
- Współczynniki przy V w równaniu źródła do bloku C,
- Współczynniki przy i_{wy} i prądzie sterującym i_{we} do bloku D.

Cały wkład napięciowego źródła sterowanego prądem pokazany na rysunku 3.

$$\begin{array}{l}
 wy_+ \rightarrow \\
 wy_- \rightarrow \\
 we_+ \rightarrow \\
 we_- \rightarrow \\
 nowy \rightarrow \\
 nowy \rightarrow
 \end{array}
 \begin{bmatrix}
 & & & & 1 & \\
 & & & & -1 & \\
 & & & 1 & & \\
 & & & -1 & & \\
 & & 1 & -1 & & \\
 1 & -1 & & & r &
 \end{bmatrix}
 \begin{bmatrix}
 V_{wy+} \\
 V_{wy-} \\
 V_{we+} \\
 V_{we-} \\
 I_{we} \\
 I_{wy}
 \end{bmatrix}
 =
 \begin{bmatrix}
 \\
 \\
 \\
 \\
 E_{we} \\
 \end{bmatrix}$$

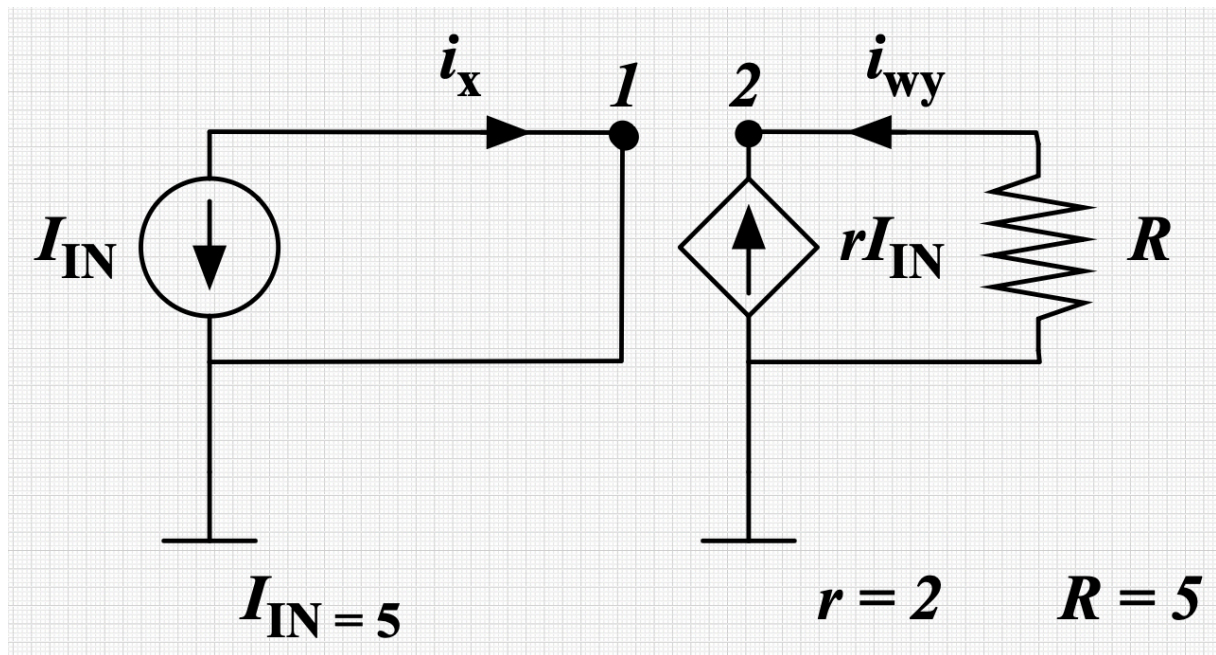
Rys. 3 - Szablon napięciowego źródła sterowanego prądem.

8. 2. Wypełnianie macierzy globalnej:

Proces wypełniania macierzy konduktancji oraz wektora wymuszeń realizowany w taki sposób:

- Zebranie węzłów i stworzenie macierzy oraz wektora z uwzględnieniem dodatkowych wierszy,
- Iteracja po elementach, czyli wywołanie funkcji `getGMatrix()` oraz `getIMatrix()` każdego elementu i sumowanie macierzy i wektorów,
- Dodatkowe sprawdzenie poprawności utworzonej macierzy, jak na przykład sprawdzenie czy nie brakuje równań.

Wynik takiego procesu pokazany na rysunkach 4, 5 i 6.



Rys. 4 - Przykładowy układ z źródłem napięciowym sterowanym prądem

```
Matrix rows: 4, columns: 4, id: 33FE9144-2715-4577-897D-1BDAF5CC3A48: [
  0.0, 0.0, 1.0, 0.0,    // ← 1
  0.0, 0.2, 0.0, 1.0,    // ← 2
  1.0, 0.0, 0.0, 0.0,    // ← ix
  0.0, 1.0, 2.0, 0.0     // ← iwy
]
```

```
Matrix rows: 4, columns: 1, id: A5163D24-E2BE-4F5C-9C2A-53F15513715E: [
  -5.0,    // ← 1
  0.0,     // ← 2
  0.0,     // ← ix
  0.0      // ← iwy
]
```

Rys. 5 - Wypełnione macierz konduktancji oraz wektor wymuszeń

```
Matrix rows: 4, columns: 1, id: 9CDCAEA3-2833-43B7-83AE-2DAD51870C63: [
  0.0,     // ← 1
  10.0,    // ← 2
  -5.0,    // ← ix
  -2.0     // ← iwy
]
```

Rys. 6 - Wynik rozwiązania układu równań

8. 3. Walidacja parametrów i zabezpieczenia w szablonach

Ponieważ szablony elementów wypełniają macierz i wektor automatycznie, błędne parametry wejściowe mogą prowadzić do niemożliwych fizycznie wyników lub macierzy osobliwych. Z tego powodu konieczna jest weryfikacja parametrów z których są tworzone elementy.

Poniżej przedstawione przykładowe zabezpieczenia dla różnych elementów.

Rezystor:

- Rezystancja ujemna - w przypadku pojawienia się ujemnej rezystancji, co powoduje niefizyczny model, zgłaszany jest błąd,
- Zerowa rezystancja - pojawienie się zerowej rezystancji może powodować do stworzenie osobliwej macierzy, co uniemożliwia rozwiązywanie układu równań. W tym przypadku zgłaszany jest błąd.

Dioda:

W ramach uwzględnić przykład nieliniowy, można przyjąć prosty model diody, czyli model Shockley'a:

$$I_D = I_S(e^{\frac{V_D}{nV_T}} - 1) \quad (21)$$

Model wymaga linearyzacji i w każdej iteracji dioda jest zastępowana odpowiednikiem liniowym: konduktancją dynamiczną g_D i źródłem prądowym I_{eq} .

Zabezpieczenia modelu diody:

- I_S nie może być ujemny ani zerowy,
- Współczynnik idealności n musi być dodatni,
- V_D przy dużych wartościach prowadzi do niemożliwych fizycznie wartości pojawiających się w wyniku obliczeń. Z tego powodu jest on ograniczany.

8. 4. Testowanie:

W ramach testowanie zostali dodane odpowiednie sprawdzania poprawnej realizacji weryfikacji i zabezpieczeń. Oprócz przypadków omówionych powyżej, również zostali dodane inne testy, jak na przykład:

- Sprawdzenie poprawności parametru, odpowiadającego dodanie nowego wierszu. W przypadku źródła napięciowego, nowy wiersz nie może być wpisywany w miejsce już istniejącego. Również krytyczne jest nie dopuszczanie tworzenia niepotrzebnych pustych wierszy,
- Sprawdzenie poprawności połączenia źródeł sterowanych. Piny wyjściowe takiego źródła nie mogą być podłączane do pinów wejściowych.

9. Obliczenie punktu pracy

9. 1. Założenia i budowa układu równań

Obliczenie punktu pracy ma na celu wyznaczenie stanu ustalonego obwodu dla stałych wymuszeń. Wynikiem tej analizy są przede wszystkim napięcia węzłowe oraz, w przypadku zastosowania ZMPW,

• dodatkowe niewiadome prądowe. Punkt pracy stanowi podstawową weryfikację poprawności zbudowanego układu równań i jest często wykorzystywany jako stan początkowy dla analizy czasowej.

W przypadku obliczania punktu pracy przyjmuje się, że wielkości nie zależą od czasu, co powoduje brak wystąpienia pochodnych. W przypadku elementów dynamicznych to oznacza:

- Kondensator nie przewodzi prądu stałego - jest rozwarciem,
- Cewka nie powoduje spadku napięcia dla prądu stałego - jest zwarciem.

W ramach realizacji bloku odpowiadającego za obliczanie punktu pracy, został użyty sposób bloku ZMPW - elementy posiadają własne szablony, a macierz konduktancji i wektor niewiadomych powstają poprzez ich sumowanie. Różnica polega na uwzględnieniu wyłącznie wkładów elementów statycznych, jak na przykład rezystor czy elementy sterowane, oraz zamienniki kondensatora i cewki.

9. 2. Walidacja i obsługa błędów

Obliczanie punktu pracy, w którym najczęściej ujawniają się błędy logiczne wynikające z niepoprawnej realizacji zabezpieczeń i/lub weryfikacji parametrów.

Przykłady sytuacji problemowych:

- Niepoprawne wymiary wykładów. Wykrycie macierzy i/lub wektora o niepoprawnych rozmiarach wskazuje na błędną implementację zabezpieczeń parsera netlisty i klasy Matrix(),
- Macierz osobliwa, brak możliwości rozwiązania. Pojawienie podobnego błędu wskazuje na możliwą błędną implementację zabezpieczeń:
 - Wykrycie wiszącego węzła przez parser netlisty,
 - Wykrycie niepoprawnych parametrów w szablonach,
 - Wykrycie braku sprawdzenia poprawnego podłączenia elementu.
- Niepoprawne wyniki numeryczne, czyli NaN/Inf. Taki błąd wskazuje na możliwy brak zabezpieczenia pewnego parametru szablonu, czy na nie poprawność opisu układu wejściowego.

9. 3. Testowanie

W ramach testowania modułu odpowiadającego za obliczanie punktu pracy, zostały sprawdzone wyniki rozwiązywania prostych układów:

- Źródło napięciowe oraz prosty dzielnik,
- Źródło prądowe oraz dzielnik napięciowy składający się z szeregowo i równolegle połączonych rezystorów.

10. Analiza czasowa

10. 1. Założenia

Analiza czasowa służy do wyznaczenia odpowiedzi układu w funkcji czasu dla wymuszeń zmiennych w czasie oraz dla obecności elementów magazynujących energię (kondensatorów i cewek). W przeciwieństwie do punktu pracy, gdzie rozwiązywany jest jeden układ równań, w analizie transient

rozwiązywana jest sekwencja układów liniowych w kolejnych chwilach t_k , przy zadanym kroku czasowym h .

Parametry analizy czasowej:

- Czas startu i końca symulacji - t_{start} , t_{stop} ,
- Krok czasowy - h ,
- Wartości początkowe. Używane zamiast wartości wynikających z obliczania punktu pracy.

Podczas implementowania zostało przyjęte, że stan początkowy dla analizy czasowej wynika z punktu pracy, co ogranicza ryzyko pojawienia się błędów na pierwszym kroku symulacji.

10. 2. Dyskretyzacja elementów dynamicznych

Ponieważ elementy dynamiczne prowadzą do układów różniczkowych, konieczne jest ich dyskretyzowanie, co pozwala w każdym kroku czasu przedstawić taki element jako równoważny wkład do macierzy konduktancji i wektorów.

Kondensator

Dla kondensatora obowiązuje (22)

$$i_c = C \frac{dv}{dt} \quad (22)$$

Po dyskretyzacji interpolacyjną metodą Eulera prąd kondensatora w kroku k można zapisać w postaci:

$$i_{c(k)} = G_c v_{(k)} + i_{c(k-1)} \quad (23)$$

gdzie:

- $G_c = \frac{C}{h}$ jest konduktancją zastępczą dodawaną do macierzy konduktancji,
- $i_{c(k-1)}$ jest wartością prądu uzyskaną w poprzednim kroku symulacji, przenoszona do wektora wymuszeń.

Wtedy kondensator zachowuje się jak:

- Rezystor o konduktancji G_c dla macierzy konduktancji,
- Źródło prądowe zależne od poprzedniego kroku, dla wektora wymuszeń.

Cewka

Dla cewki obowiązuje (24)

$$v_L = L \frac{di}{dt} \quad (24)$$

Jak i w przypadku kondensatora, cewka w każdym kroku czasowym jest zastąpiona innymi elementami. W odróżnieniu od kondensatora, cewka dodatkowo dodaje nowy wiersz do macierzy konduktancji i wektorów.

10. 3. Konstrukcja układu równań w kolejnych krokach czasowych

W każdym kroku czasowym t_k budowany jest układ równań postaci (25)

$$A_k x_k = z_k \quad (25)$$

gdzie:

- A_k jest macierzą konduktancji w kroku k ,
- x_k zawiera napięcia węzłowe i dodatkowe prądy ZMPW, czyli wektor niewiadomych,
- z_k reprezentuje wymuszenia niezależne w chwili t_k oraz wkłady z poprzednich kroków.

W ramach realizacji projektu zostało wybrane podejście, polegające na pełne wypełnienie macierzy konduktancji i wektora wymuszeń w każdym kroku czasowym. W porównaniu do podejścia polegającym na aktualizacji części dynamicznej bez zmian części statycznej, jest ono prostsze i bardziej odporne na błędy logiki.

10. 4. Pętla symulacji czasowej

Realizacja analizy czasowej:

- Ustalenie parametrów t_{start} , t_{stop} , h oraz wyznaczenie początkowego punktu pracy,
- Dla każdego kroku:
 - Wyznaczenie bieżącego czasu t_k ,
 - Przygotowanie wymuszeń zależnych od czasu,
 - Wyznaczenie wkładów elementów dynamicznych z poprzednich kroków czasowych,
 - Wypełnienie A_k i z_k przy użyciu szablonów elementów,
 - Rozwiązywanie układu postaci (25),
 - Zapisanie wyników,
 - Aktualizacja stanów elementów dynamicznych, potrzebnych dla kroku $k + 1$,
- Zakończenie symulacji po osiągnięciu t_{stop} lub wykryciu błędu.

Wyniki są zapisywane w postaci napięć/prądów w funkcji czasu, co umożliwia dalsze analizowanie oraz wyświetlenie wyników w postaci przebiegów.

10. 5. Stabilność i zabezpieczenia

Analiza czasowa jest bardzo wrażliwa na problemy numeryczne. Z tego powodu konieczne jest dodanie zabezpieczeń.

Dodane zabezpieczenia:

- Wykrywanie wartości niemożliwych fizycznie. W przypadku gdy w wektorze niewiadomych po rozwiązywaniu układu równań pojawiają się wartości NaN/Inf, zostaje zgłoszony błąd,
- Ograniczenie modeli nieliniowych. Przykładem takiego ograniczenia może występować dodanie do modeli diody minimalnej konduktancji, czy ograniczenie wpływu napięcia odkładającego się na diodzie,
- Zgłoszenie błędu w przypadku braku rozwiązania układu równań.

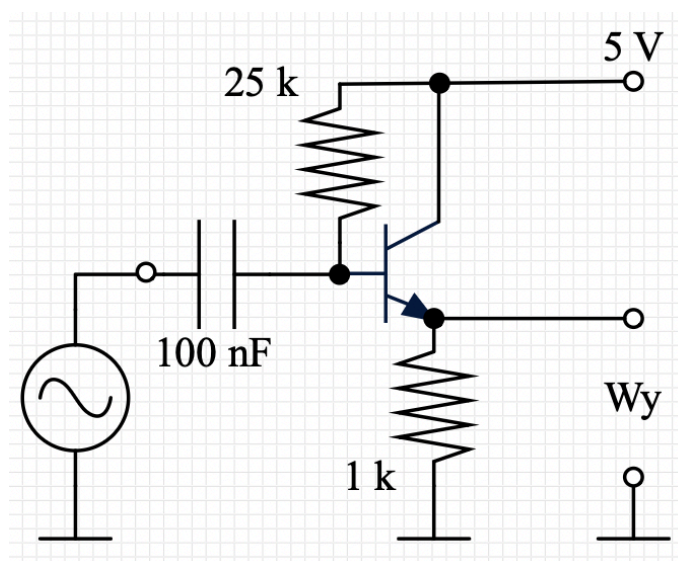
10. 6. Prezentacja wyników i format danych wyjściowych

Wyniki analizy czasowej są zapisywane jako tablica wektorów niewiadomych. Ponieważ w ramach realizacji projektu nie byli symulowany duże układy, zapisywanie których w taki sposób nie byłoby z powodu ograniczonej pamięci, nie była konieczna implementacja bardziej wydajnego przechowywania wyników poszczególnych kroków czasowych.

Również takie podejście ułatwia proces wyświetlania wyników w postaci przebiegów oraz ich zapisywanie w pamięci komputera.

10. 7. Testowanie

Testy analizy czasowej zostali prowadzone na podstawie symulacji układu wtórnika emiterowego pokazanego na rysunku 7. Taki układ posiada element nieliniowy, czyli tranzystor oraz generuje łatwe do sprawdzenia przebiegi. Wyniki symulacji pokazane na rysunkach 8 i 9.



Rys. 7 - Wtórnik emiterowy.

Inne parametry układu: składowa stała sygnału wynosi zero, sygnał sinusoidalny, amplituda sygnału wejściowego 2 V oraz częstotliwość sygnału wejściowego wynosi 1 kHz.

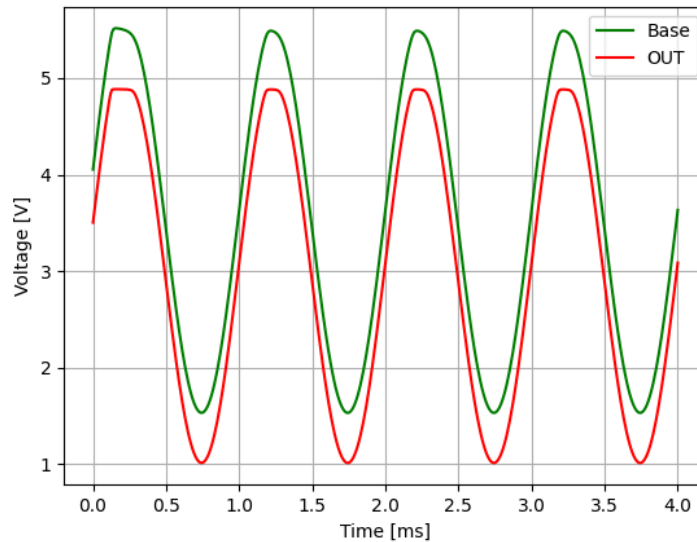
Weryfikacje obejmują:

- Obecność oczekiwanego przesunięcia napięciowego rzędu 0.7 V,
- Brak oscylacji numerycznych,

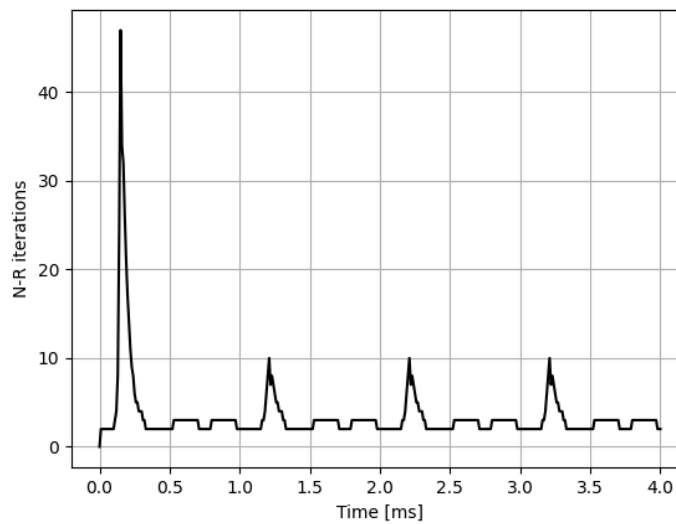
- Obecność wpływu zmiany kroku h .

Parametry symulacji:

- Czas symulacji od 0 s do 4 ms,
- Krok czasowy 10 μ s.



Rys. 8 - Napięcia na bazie i wyjściu wtórnika.

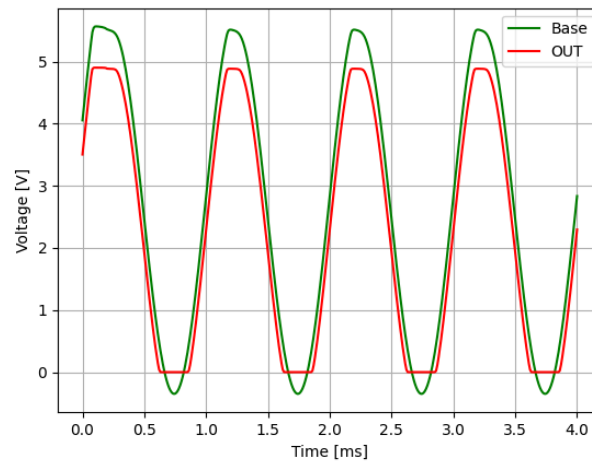


Rys. 9 - Liczba iteracji w funkcji czasu.

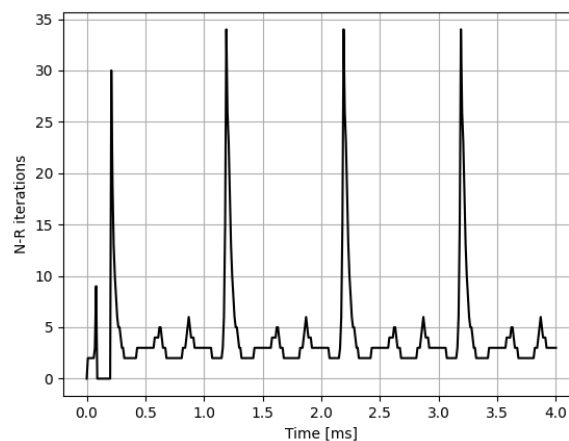
Zmiany parametrów elementów wtórnika i symulacji:

- Rysunki 10 i 11 pokazują wyniki symulacji po zwiększeniu amplitudy sygnału wejściowego do 3 V,

- Rysunki 12 i 13 pokazują wyniki symulacji po zwiększeniu kroku czasowego h .



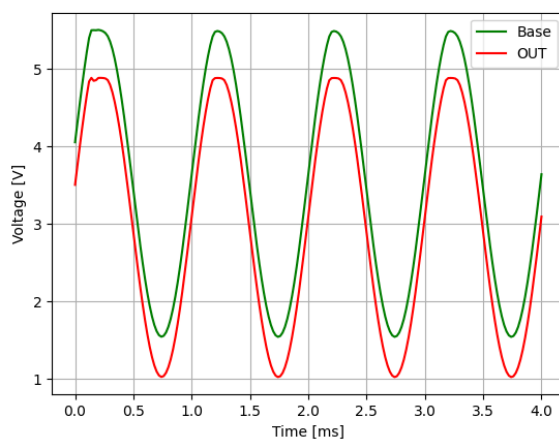
Rys. 10 - Napięcia na bazie i wyjściu wtórnika po zwiększeniu amplitudy sygnału wejściowego.



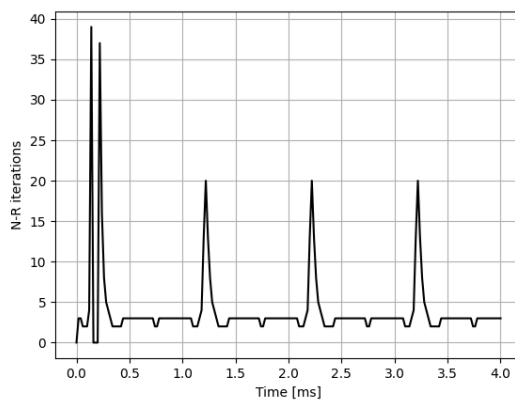
Rys. 11 - Liczba iteracji w funkcji czasu po zwiększeniu amplitudy sygnału wejściowego.

Wyniki pokazują poprawność zachowania aplikacji podczas symulacji wtórnika po zwiększeniu amplitudy sygnału wejściowego. Również taka wersja wtórnika wymaga dużo większej liczby iteracji N-R w momentach obliczeń obciążonych wierzchołków sinusoidy.

W przypadku zwiększenia kroku czasowego na rysunku 12 widać wprowadzenie oscylacji numerycznych oraz wygładzenie obciążonych wierzchołków sinusoidy oraz na rysunku 13 widać wzrost liczby iteracji N-R.



Rys. 12 - Napięcia na bazie i wyjściu wtórniaka po zwiększeniu kroku czasowego.



Rys. 13 - Liczba iteracji w funkcji czasu po zwiększeniu kroku czasowego.

11. Podsumowanie

W ramach pracy zostały zrealizowane takie elementy aplikacji:

- Parser netlisty
- Implementacja macierzy
- ZMPW oraz szablony elementów
- Obliczanie punktu pracy
- Analiza czasowa

Zrealizowane elementy pozwalają na analizę analogowych układów elektronicznych. Dalszy rozwój projektu może polegać na:

- Zoptymalizowaniu implementacji macierzy, wczytania pliku wejściowego oraz przechowywania wyników symulacji czasowej,
- Dodanie szablonów elementów,
- Dodanie metod całkowania umożliwiających lepszą analizę dla poszczególnych obwodów.

Bibliografia

- [1] "NASA, An Annotated Bibliography of Computer-Aided Circuit Analysis and Design, 1965", [Online]. Dostępne na: <https://ntrs.nasa.gov/api/citations/19680010413/downloads/19680010413.pdf>
- [2] "L. W. Nagel, D. O. Pederson, Simulation Program with Integrated Circuit Emphasis(SPICE), 1973", [Online]. Dostępne na: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1973/ERL-m-382.pdf>
- [3] "KB: Run Altium on non-Windows platform", [Online]. Dostępne na: <https://www.altium.com/documentation/knowledge-base/altium-designer/run-altium-on-non-windows-platform>
- [4] "Paul Hübner, Andong Hu, Ivy Peng, Stefano Markidis, Apple vs Oranges: Evaluation Apple Silicon M-Series SoCs for HPC performance and Efficiency, 2025", [Online]. Dostępne na: <https://arxiv.org/html/2502.05317v1>
- [5] "WikipediA, Ngspice", [Online]. Dostępne na: <https://en.wikipedia.org/wiki/Ngspice>
- [6] "WikipediA, Kirchhoff's circuit laws", [Online]. Dostępne na: https://en.wikipedia.org/wiki/Kirchhoff%27s_circuit_laws
- [7] "A system approach to circuits, measurements and control", [Online]. Dostępne na: <https://uom.lk/sites/default/files/elect/files/ee321s3.3.pdf>
- [8] "WikipediA, Nodal Analysis", [Online]. Dostępne na: https://en.wikipedia.org/wiki/Nodal_analysis
- [9] "WikipediA, Modified nodal analysis", [Online]. Dostępne na: https://en.wikipedia.org/wiki/Modified_nodal_analysis
- [10] "Paweł Plaskura, Zaawansowane metody symulacji układów elektronicznych, 2013", [Online]. Dostępne na: <http://epub.aiva.pl/filerel.php/978-83-937245-0-5.epub/978-83-937245-0-5.pdf>
- [11] "Apple, Swift Documentation, Solving systems of linear equations with LAPACK", [Online]. Dostępne na: <https://developer.apple.com/documentation/Accelerate/solving-systems-of-linear-equations-with-lapack>

Wykaz symboli i skrótów

ECAP - Electronic Circuit Analysis Program:

SPICE - Simulation Program with Integrated Circuit Emphasis: oprogramowanie do modelowania oraz symulacji funkcjonowania elektronicznych układów analogowych i cyfrowych.

ZMPW - Zmodyfikowana Metoda Potencjałów Węzłowych: metoda analizy obwodów elektrycznych na podstawie praw Kirchhoffa i charakterystyk napięciowo-prądowych elementów elektrycznych.

KCL - Kirchhoff Current Law: Pierwsze prawo Kirchhoffa, czyli prądowe prawo Kirchhoffa.

KVL - Kirchhoff Voltage Law: Drugie prawo Kirchhoffa, czyli napięciowe prawo Kirchhoffa.

NaN - Not a number: liczba o niezdefiniowanej lub niereprezentowalnej wielkości.

Inf - Infinity: reprezentacja nieskończoności.

N-R - Metoda Newtona-Raphsona: algorytm iteracyjny prowadzący do wyznaczenia przybliżonej wartości miejsca zerowego funkcji jednej zmiennej lub wielu zmiennych.

Spis rysunków

Rys. 1	Porównanie szybkości rozwiązywania układu równań.....	16
Rys. 2	Szablon rezystora.....	18
Rys. 3	Szablon napięciowego źródła sterowanego prądem.....	19
Rys. 4	Przykładowy układ z źródłem napięciowym sterowanym prądem.....	20
Rys. 5	Wypełnione macierz konduktancji oraz wektor wymuszeń.....	20
Rys. 6	Wynik rozwiązania układu równań.....	20
Rys. 7	Wtórnik emiterowy.....	25
Rys. 8	Napięcia na bazie i wyjściu wtórника.....	26
Rys. 9	Liczba iteracji w funkcji czasu.....	26
Rys. 10	Napięcia na bazie i wyjściu wtórника po zwiększeniu amplitudy sygnału wejściowego.....	27
Rys. 11	Liczba iteracji w funkcji czasu po zwiększeniu amplitudy sygnału wejściowego.....	27
Rys. 12	Napięcia na bazie i wyjściu wtórника po zwiększeniu kroku czasowego.....	28
Rys. 13	Liczba iteracji w funkcji czasu po zwiększeniu kroku czasowego.....	28